Algorithm Design: Practical Techniques and Problem-Solving Strategies

Dr. Mukesh Kumar
Associate Professor
Department of Engineering & Technology
Gurugram University, Gurugram, Haryana

Preface

This book focuses on the practical aspects of algorithm design: how to approach problems systematically, choose appropriate techniques, and implement solutions effectively. Rather than emphasizing theoretical analysis alone, we combine mathematical rigor with practical problem-solving strategies.

The material is organized around design paradigms and problem-solving approaches, with extensive examples demonstrating how to apply techniques to diverse problem classes. Students will learn both the art of recognizing problem structure and the science of rigorous design.

Contents

1 Module 1: Problem Solving and Algorithm Fundamentals 1

1.1 Algorithm Design Process 1
1.2 Problem Types 2
1.3 Algorithm Correctness 3
1.4 Brute Force Approach 3
1.5 Exhaustive Search 4
1.6 Performance Profiling 6

2 Module 2: Decrease-and-Conquer 9

2.1 Decrease by Constant 9
2.2 Decrease by Constant Factor 9
2.3 Decrease by Variable 10
2.4 BFS Applications 12
2.5 Interpolation Search 13

3 Module 3: Transform-and-Conquer 15

3.1 Presorting 15
3.2 Gaussian Elimination 16
3.3 Balanced Search Trees 16
3.4 Heaps and Heap Sort 17
3.5 Horner's Rule 18
3.6 Problem Reduction 18

4 Module 4: Space-Time Tradeoffs and String Algorithms 21

4.1 Input Enhancement 21
4.2 Prestructuring 22
4.3 Dynamic Programming with Space Optimization . 23
4.4 String Matching Algorithms 23
4.5 Tries and Suffix Arrays 25
4.6 Algorithm Complexity Comparison 26

Module 1: Problem Solving and Algorithm Fundamentals

1.1 Algorithm Design Process

Designing algorithms is a structured process with distinct phases:

Problem Analysis: Understand the problem precisely. Identify inputs, outputs, constraints, and success criteria. Clarify ambiguities with stakeholders. Estimate input size: this determines feasible complexity.

Example: "Sort array of numbers." More precise: "Given array of n 32-bit integers, sort in non-decreasing order. $n \leq 10^6$. Minimize time, secondary objective: minimize space."

Algorithm Design: Develop solution strategy. Consider multiple approaches: brute force, divide and conquer, dynamic programming, greedy. Sketch high-level approach before detailed implementation.

Example: For sorting, consider insertion sort ($O(n^2)$), merge sort ($O(n \log n)$), quicksort ($O(n \log n)$ average). Given $n \leq 10^6$, merge sort guarantees performance.

Correctness Proof: Verify algorithm correctness. Methods: loop invariants, mathematical induction, case analysis. Informal arguments: trace on examples, reason about algorithm behavior.

Example: For merge sort, argue that merging two sorted arrays produces sorted result (true by construction), so recursion produces sorted result.

Complexity Analysis: Analyze time and space complexity. Determine best, average, worst cases. Compare candidates: theoretically (asymptotic complexity) and empirically (constant factors, real-world performance).

Example: Merge sort is $O(n \log n)$ always; quicksort is $O(n \log n)$

average but $O(n^2)$ worst case with poor pivot selection.

Implementation: Code the algorithm in chosen language. Use appropriate data structures. Follow best practices: readable code, error handling, testing.

Example: Implement merge sort with helper function for merging, recursively divide, ensure no off-by-one errors.

1.2 Problem Types

Different problem types require different approaches:

Sorting: Order elements by key. Standard: Merge sort or quicksort $O(n \log n)$. Specialized: Counting sort (limited range), Radix sort (small digits).

Searching: Find element or position in data. Array: Binary search $O(\log n)$ in sorted array, linear $O(n)$ unsorted. Graph: BFS/DFS $O(V + E)$. String: Pattern matching $O(n + m)$ with KMP.

String Processing: Pattern matching, substring search, longest common subsequence, edit distance. Algorithms: KMP, Boyer-Moore, Rabin-Karp, dynamic programming, suffix trees.

Combinatorial: Select subset, arrangement, permutation. Examples: knapsack, assignment problem, traveling salesman. Approaches: brute force (exponential), dynamic programming, greedy approximation.

Graph Problems: Connectivity, paths, flows, matching. Algorithms: BFS, DFS, Dijkstra, Bellman-Ford, Floyd-Warshall, maximum flow.

Geometric: Points, shapes, convex hulls, intersections. Algorithms: convex hull (Graham scan), point location, Voronoi diagrams.

Numerical: Root finding, integration, optimization. Approaches: Newton's method, gradient descent, numerical integration.

1.3 Algorithm Correctness

Proving algorithms correct is essential. Methods:

Loop Invariant: Assertion that holds before loop starts, after each iteration, and after loop terminates. Prove: initialization (true before loop), maintenance (if true before iteration, true after), termination (after loop, invariant implies correctness).

Example: Insertion sort maintains invariant "array $[1..i]$ is sorted." Initially $i = 1$, trivially sorted. After inserting $A[i + 1]$ into correct position, $[1..i + 1]$ remains sorted. After loop ($i = n$), entire array is sorted.

Induction: For recursive algorithms or problems reducible to subproblems. Prove base case ($n = 1$) and inductive step (assume true for k, prove for $k + 1$).

Example: Merge sort correctness: Base case: array of size 1 is sorted. Inductive step: recursively sort halves (sorted by induction), merge produces sorted result (merging sorted arrays yields sorted result).

Case Analysis: Prove all cases lead to correct result.

Example: Quicksort correctness: After partition around pivot, left subarray has elements $<$ pivot, right has elements $>$ pivot. Recursively sorting each yields overall sorted array.

Mathematical Argument: Reason about algorithm properties.

Example: Binary search invariant: target in range $[low, high]$. Each iteration halves range, eventually singleton or empty.

1.4 Brute Force Approach

Brute force enumerates all possibilities and selects best. Simple, often correct, but inefficient.

String Matching Brute Force

```
Algorithm BruteForceMatch(text, pattern)
  for i = 0 to len(text) - len(pattern) do
```

```
    match = true
    for j = 0 to len(pattern) - 1 do
      if text[i+j] != pattern[j] then
        match = false
        break
    if match then
      return i
  return -1
```

Time: $O(nm)$ where $n =$ text length, $m =$ pattern length. For $n = m = 10^6$, about 10^{12} operations: too slow.

Optimization: Use Boyer-Moore to skip positions based on mismatches. Practical performance: $O(n/m)$ best case.

Closest Pair Problem

Find two points with minimum distance.

Brute force: Check all $\binom{n}{2} = O(n^2)$ pairs.

```
Algorithm ClosestPair(points)
  minDist = infinity
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      dist = distance(points[i], points[j])
      minDist = min(minDist, dist)
  return minDist
```

Time: $O(n^2)$.

Optimization: Divide and conquer (sort by x-coordinate, divide, recursively solve, merge strip). Time: $O(n \log n)$.

1.5 Exhaustive Search

Exhaustive search explores entire solution space: subsets, permutations, arrangements.

Subset Enumeration

Generate all 2^n subsets of set $\{1, 2, \ldots, n\}$.

```
Algorithm SubsetEnumeration(n)
  for mask = 0 to 2^n - 1 do
    subset = []
    for i = 0 to n-1 do
      if (mask & (1 << i)) != 0 then
        subset.append(i+1)
    process(subset)
```

Time: $O(n \cdot 2^n)$ (enumerate subsets + process each).

For $n > 20$, exhaustive is infeasible. Use optimization: branch and bound (prune subsets exceeding bound), dynamic programming (store optimal solutions to subproblems), approximation (find near-optimal in poly time).

Permutation Generation

Generate all $n!$ permutations.

```
Algorithm Permutations(arr, start, end)
  if start == end then
    process(arr)
  else
    for i = start to end do
      swap(arr[start], arr[i])
      Permutations(arr, start+1, end)
      swap(arr[start], arr[i])
```

Time: $O(n \cdot n!)$ (enumerate permutations + process each).

For $n > 10$, exhaustive is infeasible. Use heuristics: local search (swap adjacent elements), genetic algorithms (population-based search).

1.6 Performance Profiling

Understanding actual performance requires measurement, not just asymptotic analysis.

Constant factors: $O(n)$ algorithm with constant 100 is slower than $O(n \log n)$ with constant 1 for small n.

Cache effects: Sequential access exploits cache; random access misses cache repeatedly. Algorithms with good cache locality (e.g., merge sort over quicksort) may be faster despite same asymptotic complexity.

Instruction-level parallelism: Modern CPUs execute multiple instructions per cycle. Branch mispredictions, cache misses stall execution. Data-parallel algorithms (SIMD) vectorize: process multiple elements per instruction.

I/O bounds: For external data (disk, network), I/O dominates computation. Minimize I/O: read sequentially, minimize roundtrips, buffer results.

Example profiling approach:

```
for each input_size in [1000, 10000, 100000]:
  for each algorithm in [brute_force, optimized]:
    timer.start()
    repeat 10 times:
      result = algorithm(input)
    elapsed = timer.stop()
    average_time = elapsed / 10
    print(algorithm, input_size, average_time)
```

Compare measured performance to theoretical predictions. Identify constants and optimize accordingly.

Worked Example 1: Algorithm Complexity Proof

Prove insertion sort is $O(n^2)$.

Loop invariant: After i iterations, $A[1..i]$ is sorted.

Base: $i = 1$, $A[1]$ is trivially sorted.

Maintenance: Assume $A[1..i]$ is sorted after i iterations. In iteration $i+1$, we insert $A[i+1]$ into correct position, maintaining sorted order of $A[1..i+1]$.

Termination: After n iterations, $A[1..n]$ is sorted.

Time analysis: Outer loop n iterations. Inner loop: iteration i inserts $A[i]$ into sorted prefix, up to i comparisons. Total: $\sum_{i=1}^{n} i = O(n^2)$.

Worked Example 2: Brute Force vs. Optimized

For closest pair with $n = 10^4$ points:

Brute force: $n^2/2 = 5 \times 10^7$ comparisons. Assuming 1 billion comparisons/second, about 50ms.

Divide and conquer: $n \log n = 1.3 \times 10^5$ distance calculations. 1000x faster in theory, similar in practice (high constant factors).

Practical: Brute force is fastest for moderate n due to simplicity and cache locality. Divide and conquer wins for large n.

Worked Example 3: Exhaustive Search for TSP

Traveling Salesman with $n = 10$ cities:

All tours: $(n-1)!/2 = 181440$ (account for direction and starting city).

Brute force enumeration: Feasible in seconds.

For $n = 20$: $(20-1)!/2 \approx 6 \times 10^{16}$ tours. Infeasible.

Solution: Use branch and bound (prune tours exceeding best found), heuristics (nearest neighbor, 2-opt), approximation (2-approximation via MST).

Review Questions

1. Describe the algorithm design process with phases and deliverables.

2. What is a loop invariant and how does it prove correctness?

3. Compare problem types and identify appropriate algorithms for each.
4. When is brute force acceptable vs. requiring optimization?
5. Explain the difference between asymptotic complexity and constant factors.
6. How would you profile algorithm performance empirically?
7. For string matching, why is brute force $O(nm)$ and when is optimization necessary?
8. Give an example of exhaustive search and explain when it becomes infeasible.

Module 2: Decrease-and-Conquer

2.1 Decrease by Constant

Decrease by constant reduces problem size by fixed amount each step.

Insertion Sort

Insertion sort decreases by 1: solve $(n-1)$-element problem, then insert n-th element.

Time: $O(n^2)$ worst case, $O(n)$ best case (nearly sorted). Good for small or nearly-sorted arrays.

Graph Traversal

BFS and DFS decrease by 1 vertex each step.

```
Algorithm DFS(v):
  visited[v] = true
  for each neighbor u of v:
    if not visited[u]:
      DFS(u)
```

Recursion depth: $O(V)$. Total operations: $O(V+E)$.

2.2 Decrease by Constant Factor

Decrease by constant factor divides problem size by fixed ratio.

Binary Search

For sorted array, repeatedly divide in half.

```
Algorithm BinarySearch(A, target, low, high):
  if low > high:
    return -1
  mid = floor((low + high) / 2)
  if A[mid] == target:
    return mid
  else if A[mid] < target:
    return BinarySearch(A, target, mid+1, high)
  else:
    return BinarySearch(A, target, low, mid-1)
```

Recurrence: $T(n) = T(n/2)+O(1)$, solving to $T(n) = O(\log n)$.

Exponentiation by Squaring

Compute x^n in $O(\log n)$ multiplications:
$x^n = (x^{n/2})^2$ if n even, $x \cdot x^{n-1}$ if n odd.

```
Algorithm FastPower(x, n):
  if n == 0:
    return 1
  if n is even:
    half = FastPower(x, n/2)
    return half * half
  else:
    return x * FastPower(x, n-1)
```

Time: $O(\log n)$ multiplications.

2.3 Decrease by Variable

Problem size reduces by varying amounts.

Euclidean GCD Algorithm

Find greatest common divisor via repeated subtraction/division:

```
Algorithm GCD(a, b):
  if b == 0:
    return a
  return GCD(b, a mod b)
```

Complexity: $O(\log \min(a, b))$ divisions (Fibonacci sequence property).

Example: $\gcd(1071, 462)$. $1071 = 462 \cdot 2 + 147$. $462 = 147 \cdot 3 + 21$. $147 = 21 \cdot 7 + 0$. So $\gcd(1071, 462) = 21$.

DFS for Cycle Detection

DFS with back edge detection identifies cycles.

```
Algorithm HasCycle(v, parent):
  visited[v] = true
  for each neighbor u of v:
    if not visited[u]:
      if HasCycle(u, v):
        return true
    else if u != parent:  // Back edge
      return true
  return false
```

Time: $O(V + E)$.

Topological Sort

Order DAG vertices so every edge goes forward.

```
Algorithm TopologicalSort(G):
  stack = empty
  for each vertex v:
```

```
    if not visited[v]:
      DFSTopSort(v, stack)
  return stack (reversed)

Algorithm DFSTopSort(v, stack):
  visited[v] = true
  for each neighbor u of v:
    if not visited[u]:
      DFSTopSort(u, stack)
  stack.push(v)
```

Time: $O(V + E)$. Vertices finishing first go last (reverse topological order).

2.4 BFS Applications

BFS level-order traversal enables shortest path in unweighted graphs and bipartiteness testing.

Shortest Path

BFS from source s computes shortest path (fewest edges) to all vertices.

Distance $d[v]$ = level of v in BFS tree rooted at s.

```
Algorithm BFSShortestPath(G, s):
  dist[s] = 0
  queue = {s}
  while queue not empty:
    v = dequeue(queue)
    for each neighbor u of v:
      if dist[u] == undefined:
        dist[u] = dist[v] + 1
        enqueue(queue, u)
```

Time: $O(V + E)$.

Bipartiteness Testing

Graph is bipartite if vertices partition into two sets with edges only between sets (no edge within set).

BFS colors vertices: even distance from source = color 1, odd = color 2. If all edges connect different colors, bipartite.

```
Algorithm IsBipartite(G):
  color[s] = 1
  queue = {s}
  while queue not empty:
    v = dequeue(queue)
    for each neighbor u of v:
      if color[u] == undefined:
        color[u] = 3 - color[v]  // Flip color
        enqueue(queue, u)
      else if color[u] == color[v]:
        return false
  return true
```

Time: $O(V + E)$.

2.5 Interpolation Search

Generalization of binary search for uniformly distributed data.

Estimate position via linear interpolation instead of middle:

$$\text{pos} = low + \frac{target - A[low]}{A[high] - A[low]} \cdot (high - low) \tag{2.1}$$

Best case: $O(\log \log n)$. Worst case: $O(n)$ (decreases by small fraction if bad estimate). Average: $O(\log \log n)$ for uniform data.

Binary search is safer (guaranteed $O(\log n)$) for general data.

Worked Example 1: Binary Search

Search for 7 in $A = [1, 3, 5, 7, 9, 11, 13]$ (length 7).

Low = 0, high = 6. Mid = 3, $A[3] = 7$. Found, return 3.

Worked Example 2: Topological Sort

DAG edges: $(A, B), (A, C), (B, D), (C, D)$.

DFS from A: Visit B, then D (finish D), back to C, then D (visited), back to A (finish A), return.

Finish order: D, B, A. Reverse: A, B, D, C (or other valid orders).

Worked Example 3: GCD via Euclidean

$\gcd(126, 84)$: $126 = 84 \cdot 1 + 42$. $84 = 42 \cdot 2 + 0$. So $\gcd(126, 84) = 42$.

Review Questions

1. Compare insertion sort and binary search as decrease-and-conquer algorithms.
2. Analyze the time complexity of binary search and exponentiation by squaring.
3. How does DFS detect cycles in undirected and directed graphs?
4. Explain topological sort and its application to DAGs.
5. When is BFS preferable to DFS and vice versa?
6. How does interpolation search improve on binary search for specific data distributions?
7. Derive the time complexity of Euclidean GCD using Fibonacci sequence properties.
8. Why is bipartiteness checkable in linear time via BFS?

Module 3: Transform-and-Conquer

3.1 Presorting

Sorting input often enables faster solutions.

Element Uniqueness

Check if array has duplicates.

Naive: Compare all pairs, $O(n^2)$.

Optimized: Sort, then scan for adjacent equals.

```
Algorithm HasDuplicate(A):
  sort(A)
  for i = 0 to len(A)-2:
    if A[i] == A[i+1]:
      return true
  return false
```

Time: $O(n \log n)$ (sorting dominates).

Anagram Detection

Detect if two strings are anagrams (rearrangements).

Naive: Compare character frequencies with hash table, $O(n)$.

Alternative: Sort both strings, compare.

```
Algorithm AreAnagrams(s1, s2):
  return sort(s1) == sort(s2)
```

Time: $O(n \log n)$ (usually slower than hash table).

Closest Pair After Sort

Sort points by x-coordinate, use divide-and-conquer within sorted range.

Time: $O(n \log n)$ after preprocessing (sort once, use for multiple queries).

3.2 Gaussian Elimination

Solve system of linear equations $A\mathbf{x} = \mathbf{b}$ via row reduction to upper triangular form.

```
Algorithm GaussianElimination(A, b):
  n = len(A)
  for col = 1 to n:
    find pivot (largest magnitude in column col)
    swap rows to move pivot to diagonal
    for row = col+1 to n:
      eliminate column col in row:
      b[row] -= (A[row][col] / A[col][col]) * b[col]
      A[row][col:] -= (A[row][col] / A[col][col]) * A[co
  back-substitute to find x
```

Time: $O(n^3)$ (main loop) + $O(n^2)$ back-substitution = $O(n^3)$.

LU decomposition: Factors $A = LU$ (lower/upper triangular). Once computed, solving $A\mathbf{x} = \mathbf{b}$ for different $\mathbf{b}$ is $O(n^2)$.

3.3 Balanced Search Trees

Balance trees to maintain $O(\log n)$ operations.

AVL Trees

AVL tree: Height balanced. After any operation, heights of left/right subtrees differ by ≤ 1.

Rotations: Single rotation or double rotation restores balance after insertion/deletion.

```
Algorithm InsertAVL(tree, key):
  insert like BST
  update heights up the tree
  for each ancestor:
    if balance factor > 1 or < -1:
      perform rotation to rebalance
```

Time: Insert/delete/search in $O(\log n)$ (balanced height).

2-3 Trees

Internal node has 2 or 3 children. All leaves at same depth.

Insertion: Add to leaf, split if has 4 children (push middle up).

Deletion: Merge/redistribute to maintain structure.

Time: Search in $O(\log n)$ (all leaves at depth $\leq \log_2 n$).

3.4 Heaps and Heap Sort

Heap: Binary tree satisfying heap property (parent $\geq$ children for max-heap).

```
Algorithm BuildHeap(A):
  for i = floor(n/2) down to 1:
    SiftDown(A, i)

Algorithm SiftDown(A, i):
  largest = i
  if 2*i <= n and A[2*i] > A[largest]:
    largest = 2*i
  if 2*i+1 <= n and A[2*i+1] > A[largest]:
    largest = 2*i+1
  if largest != i:
    swap(A[i], A[largest])
```

```
    SiftDown(A, largest)

Algorithm HeapSort(A):
  BuildHeap(A)  // O(n)
  for i = n down to 2:
    swap(A[1], A[i])
    SiftDown(A, 1)  // O(log n)
  // Total: O(n log n)
```

Build-heap is $O(n)$ (not $O(n \log n)$), proof: Most nodes are shallow, contribute little to SiftDown cost.

3.5 Horner's Rule

Evaluate polynomial $p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$ efficiently.

Naive: $a_0 + a_1 x + a_2 x \cdot x + \cdots$ requires many multiplications.

Horner's rule:

$$p(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + x \cdot a_n) \cdots)) \quad (3.1)$$

```
Algorithm HornersRule(a, x):
  result = 0
  for i = n down to 0:
    result = result * x + a[i]
  return result
```

Time: $O(n)$ multiplications (optimal, can't do better).

3.6 Problem Reduction

Reduce problem to known solvable problem.

Example: Bipartite matching via maximum flow. Model: create source connected to left vertices, right connected to sink, edges between left/right, capacity 1. Max flow = maximum matching.

Worked Example 1: Gaussian Elimination

System: $2x + y = 5$, $4x + 3y = 11$.

Matrix form: $\begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 5 \\ 11 \end{bmatrix}$.

Row 2 -= 2 * Row 1: $\begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 5 \\ 1 \end{bmatrix}$.

Back-substitute: $y = 1$, $2x + 1 = 5$, so $x = 2$.

Worked Example 2: Heap Sort

Array: $[5, 3, 8, 1, 9]$. Build heap (max-heap): $[9, 5, 8, 1, 3]$.

Extract: Swap 9 with 3, sift down. Heap: $[8, 5, 3, 1]$, extract 8. Continue.

Sorted: $[1, 3, 5, 8, 9]$.

Worked Example 3: Horner's Rule

Evaluate $p(x) = 2 + 3x + x^2$ at $x = 2$.

Horner: result = 0. $i = 2$: result = $0 \cdot 2 + 1 = 1$. $i = 1$: result = $1 \cdot 2 + 3 = 5$. $i = 0$: result = $5 \cdot 2 + 2 = 12$.

Direct: $2 + 3(2) + 2^2 = 2 + 6 + 4 = 12$. Correct.

Review Questions

1. When does presorting improve algorithm efficiency?
2. Explain the Gaussian elimination process and LU decomposition.
3. How do AVL trees maintain balance and what is the complexity of operations?
4. Analyze the time complexity of BuildHeap and prove $O(n)$ bound.
5. Why is Horner's rule optimal for polynomial evaluation?

6. Give an example of problem reduction and the resulting algorithm.
7. Compare 2-3 trees and AVL trees in terms of balance guarantee.
8. How would you detect cycles using presorting?

Module 4: Space-Time Tradeoffs and String Algorithms

4.1 Input Enhancement

Preprocess input to enable faster processing.

Counting Sort

Assumes integers in range $[0, k]$. Preprocess: count occurrences in $O(n + k)$.

```
Algorithm CountingSort(A, k):
  count = [0] * (k+1)
  for x in A:
    count[x] += 1
  for i = 1 to k:
    count[i] += count[i-1]
  result = [0] * len(A)
  for x in A (backwards):
    result[count[x]-1] = x
    count[x] -= 1
  return result
```

Time: $O(n + k)$. Linear time when $k = O(n)$.

Hashing

Hash function maps data to indices. Preprocessing: insert n elements into hash table.

Hash table enables: search, insert, delete in $O(1)$ expected time (with good hash function and low load factor).

Hash function properties: uniform distribution (minimize collisions), fast computation, deterministic.

Common: modular hashing $h(x) = x \mod m$, polynomial hashing $h(x) = (a_0 + a_1 b + a_2 b^2 + \cdots) \mod m$.

4.2 Prestructuring

Build data structures for efficient querying.

B-Trees

Tree structure optimized for disk access. Node size matches disk block. Branching factor b (number of children per node).

Tree depth: $O(\log_b n)$. Search/insert/delete: $O(\log_b n)$ disk accesses.

Example: $b = 1000$ (typical for disk), depth 4 accesses $1000^4 = 10^{12}$ elements.

Hash Tables with Chaining

Hash table: array of buckets. Collisions: chain elements in bucket via linked list.

```
Algorithm Insert(key, value):
  h = hash(key) % table_size
  insert into table[h] chain

Algorithm Search(key):
  h = hash(key) % table_size
  search in table[h] chain
```

Expected time: $O(1+$ load factor$)$ = $O(1 + n/m)$ where m = table size. Maintain load factor ≈ 1 via resizing.

4.3 Dynamic Programming with Space Optimization

DP stores all subproblem solutions. Can optimize space by noting only recent rows/columns needed.

0/1 Knapsack Space Optimization

Standard DP: $O(nW)$ space for $dp[n][W]$ table.

Optimized: Use rolling array. $dp[w]$ = max value with weight limit w. Process items, update array in-place (from right to left to avoid using updated values).

```
Algorithm KnapsackOptimized(items, W):
  dp = [0] * (W+1)
  for item in items:
    for w = W down to item.weight:
      dp[w] = max(dp[w],
                  dp[w - item.weight] + item.value)
  return dp[W]
```

Time: $O(nW)$. Space: $O(W)$ (vs. $O(nW)$).

4.4 String Matching Algorithms

Naive String Matching

Compare pattern at each position.

Time: $O(nm)$ where $n =$ text length, $m =$ pattern length. Too slow for large text/pattern.

Rabin-Karp Algorithm

Use polynomial rolling hash to speed up comparison.

Hash pattern: $h_p = (a_0 + a_1 b + a_2 b^2 + \cdots + a_{m-1} b^{m-1}) \bmod q$.

Hash text substrings: h_t = hash of current window. Update in $O(1)$ via rolling hash:

$$h_{\text{next}} = (h_{\text{current}} - \text{text}[j] \cdot b^{m-1}) \cdot b + \text{text}[j+m] \mod q \quad (4.1)$$

If $h_p = h_t$, verify match (hash collision possible).

```
Algorithm RabinKarp(text, pattern):
  hp = hash(pattern)
  ht = hash(text[0:m])
  for i = 0 to n-m:
    if hp == ht:
      if text[i:i+m] == pattern:
        return i
    ht = (ht - text[i]*b^(m-1)) * b + text[i+m]
  return -1
```

Time: $O(n+m)$ expected (assuming no hash collisions), $O(nm)$ worst case (many false positives).

Knuth-Morris-Pratt (KMP) Algorithm

Precompute failure function: longest proper prefix of pattern matching proper suffix.

Failure function guides pattern advance without backtracking.

```
Algorithm KMPMatch(text, pattern):
  failure = buildFailureFunction(pattern)
  j = 0  // pattern index
  for i = 0 to len(text)-1:
    while j > 0 and text[i] != pattern[j]:
      j = failure[j-1]
    if text[i] == pattern[j]:
      j += 1
    if j == len(pattern):
```

```
        return i - len(pattern) + 1
        j = failure[j-1]
    return -1
```

Time: $O(n + m)$ (guaranteed, no worst case).

KMP failure function for "ABABCABAB":

i	0	1	2	3	4	5	6	7	8
Pattern	A	B	A	B	C	A	B	A	B
failure	0	0	1	2	0	1	2	3	4

Table 4.1: KMP Failure Function

Boyer-Moore Algorithm

Scan pattern right-to-left. On mismatch, use bad-character and good-suffix heuristics to skip forward.

Best case: $O(n/m)$ (rarely examines each character).

Worst case: $O(nm)$.

Average case: Faster than KMP and Rabin-Karp on real text.

4.5 Tries and Suffix Arrays

Tries

Tree structure for string prefix searching. Each node has children for each character.

Search: Follow edges matching characters. Check end-of-word flag.

Insert: Create nodes as needed, mark end-of-word.

Time: $O(m)$ for string of length m. Space: $O(km)$ where k = alphabet size.

Applications: Autocomplete, IP routing (longest prefix match), spell checking.

Suffix Arrays

Array of sorted suffixes of text. Enable fast pattern searching via binary search.

Construction: Suffix array for "banana": suffixes [banana, anana, nana, ana, na, a] sorted [a, ana, anana, banana, na, nana]. Indices [5, 3, 1, 0, 4, 2].

Search: Pattern "ana" in suffixes: binary search among sorted suffixes.

Time: Build in $O(n \log n)$ via sorting (or $O(n)$ with specialized algorithm). Search: $O(m \log n)$ binary search + $O(m)$ verification.

4.6 Algorithm Complexity Comparison

Algorithm	Time	Space	Best Case	Notes
Naive	$O(nm)$	$O(1)$	$O(nm)$	Simple, no preprocessing
Rabin-Karp	$O(n+m)$	$O(1)$	$O(n+m)$	Hash-based, randomized
KMP	$O(n+m)$	$O(m)$	$O(n+m)$	Deterministic, guaranteed
Boyer-Moore	$O(nm)$	$O(km)$	$O(n/m)$	Practical, heuristics
Trie	$O(n)$	$O(km)$	$O(n)$	Fast for multiple patterns
Suffix Array	$O(n \log n)$	$O(n)$	$O(m \log n)$	Space-efficient vs. suffix tr

Table 4.2: String Matching Algorithms

Worked Example 1: Rabin-Karp

Text = "ababcab", pattern = "cab", $q = 11$, $b = 2$.

$h_p = (3 + 0 \cdot 2 + 2 \cdot 4) \mod 11 = 11 \mod 11 = 0$.

$h_t[0:2] = (0 + 1 \cdot 2 + 0 \cdot 4) \mod 11 = 2$.

$h_t[1:3] = (0 + 0 \cdot 2 + 1 \cdot 4) \mod 11 = 4$.

$h_t[2:4] = (1 + 1 \cdot 2 + 3 \cdot 4) \mod 11 = (1 + 2 + 12) \mod 11 = 15 \mod 11 = 4$.

$h_t[3:5] = ((4 - 1 \cdot 4) \cdot 2 + 0) \mod 11 = (0 \cdot 2) \mod 11 = 0$. Match! Verify "cab" = "cab". Found.

Worked Example 2: KMP Failure Function

Pattern "ABABC":

$failure[0] = 0$ (no proper prefix/suffix).

$failure[1] = 0$ (prefix "A", suffix "B", no match).

$failure[2] = 1$ (prefix "AB", suffix "AB", match "A").

$failure[3] = 2$ (prefix "ABA", suffix "ABA", match "AB").

$failure[4] = 0$ (prefix "ABAB", suffix "ABAB", no proper match with C).

Worked Example 3: Suffix Array

Text = "banana". Suffixes: [banana, anana, nana, ana, na, a]. Sorted: [a, ana, anana, banana, na, nana]. Indices: [5, 3, 1, 0, 4, 2].

Pattern "ana": Binary search. Left most at index 1 (suffix "anana"), right most at index 2 (suffix "ana" at position 3 of text, suffix "ana" at position 1). Match positions: 1, 3.

Review Questions

1. How do counting sort and hashing exemplify input enhancement?
2. Explain B-trees and when they are preferred over binary search trees.
3. Derive the rolling hash formula for Rabin-Karp algorithm.
4. Analyze the time complexity of Knuth-Morris-Pratt algorithm.
5. When is Boyer-Moore preferred over KMP in practice?
6. How do tries enable efficient multiple-pattern matching?
7. Compare suffix arrays and suffix trees in terms of space and query time.
8. Describe the space optimization technique for 0/1 knapsack DP.

Appendix: Advanced Problem-Solving Techniques

Competitive Programming Strategies

In competitions, designing and implementing correct algorithms quickly is essential.

Strategy 1: Understand the problem completely before coding. Read all constraints. Verify understanding with examples.

Strategy 2: Estimate feasible complexity. If $n \leq 10^5$, aim for $O(n \log n)$ or $O(n)$. If $n \leq 10^3$, $O(n^2)$ acceptable.

Strategy 3: Start with simplest correct solution, optimize if too slow. Premature optimization wastes time.

Strategy 4: Code defensively. Handle edge cases: empty input, single element, duplicate elements, large values.

Strategy 5: Test on examples, then submit. Don't over-think or debug excessively.

Strategy 6: Know the libraries. STL algorithms (C++), collections (Java), standard library (Python) provide efficient implementations.

Proof Techniques for Algorithms

Algorithms require proof of correctness:

Loop Invariants: Formalize property maintained each iteration. Prove initialization, maintenance, termination.

Induction: For recursive algorithms, prove base case and inductive step.

Contradiction: Assume algorithm incorrect, derive contradiction.

Exchange Argument: For greedy algorithms, show optimal solution can be transformed to greedy solution without loss.

Potential Function: Define function measuring progress. Show algorithm strictly increases potential, bounded above, so terminates.

Algorithm Design Patterns

Recognizing common patterns accelerates design:

Sliding Window: Two pointers, move right pointer to expand, left pointer to shrink. Useful for subarray/substring problems.

Fast/Slow Pointers: Two pointers at different speeds. Detect cycles in linked lists, find middle.

Multiple Pointers: Track multiple indices, merge sorted arrays, three-sum problem.

Binary Search on Answer: If checking property $P(x)$ is easy, binary search for boundary where P changes.

Two-Pass Algorithm: First pass gathers information, second pass uses information to solve.

Common Pitfalls

Avoid these mistakes:

Off-by-one errors: Loop indices, array bounds. Test carefully.

Integer overflow: For large products or sums, use long integers or modular arithmetic.

Uninitialized variables: Set to base case or -1 (depending on usage).

Not handling edge cases: Empty input, single element, duplicates.

Misunderstanding problem: Reread problem statement multiple times.

Inefficient data structures: Use appropriate structures (heap for priority queue, hash set for membership).

Resources and Further Study

Classic algorithms texts: CLRS, Knuth, Sedgewick.

Online platforms: Codeforces, TopCoder, LeetCode, HackerRank.

Websites: GeeksforGeeks, Baeldung, Medium algorithm blogs.

Research papers: For cutting-edge techniques, consult STOC, FOCS, SODA.

This book provides foundations. Mastery comes through practice: solve problems, implement algorithms, analyze complexity, optimize solutions. Develop intuition for recognizing applicable techniques. Build library of algorithmic patterns.

Algorithm design is rewarding: elegant solutions to hard problems, improving performance orders of magnitude, solving problems thought impossible. Continue learning, practicing, innovating.

Extended Examples: Complete Solutions

Problem: Longest Increasing Subsequence

Find longest subsequence of array in strictly increasing order.

Example: $[10, 9, 2, 5, 3, 7, 101, 18]$. LIS: $[2, 3, 7, 101]$ (length 4).

Approach 1: DP. $dp[i]$ = length of LIS ending at $A[i]$.

```
Algorithm LIS_DP(A):
  n = len(A)
  dp = [1] * n
  for i = 1 to n-1:
    for j = 0 to i-1:
      if A[j] < A[i]:
        dp[i] = max(dp[i], dp[j] + 1)
  return max(dp)
```

Time: $O(n^2)$. Space: $O(n)$.

Approach 2: Binary Search. Maintain array $tails$ where $tails[i] =$ smallest ending value of increasing subsequence length $i + 1$.

```
Algorithm LIS_BinarySearch(A):
  tails = []
  for x in A:
    pos = binary_search(tails, x)
    if pos == len(tails):
      tails.append(x)
    else:
      tails[pos] = x
  return len(tails)
```

Time: $O(n \log n)$. Space: $O(n)$.

Example trace: $[10, 9, 2, 5, 3, 7, 101, 18]$.

tails = [10]. After 9: [9]. After 2: [2]. After 5: [2, 5]. After 3: [2, 3]. After 7: [2, 3, 7]. After 101: [2, 3, 7, 101]. After 18: [2, 3, 7, 18]. LIS length = 4.

Problem: Merge K Sorted Lists

Merge k sorted linked lists into single sorted list.

Approach 1: Merge pairwise. Merge first two, result with third, etc.

Time: $O(nk \log k)$ (merge k lists, each merge is linear).

Approach 2: Min-heap. Maintain heap of size k (heads of each list). Extract min, add to result, push next element from that list.

```
Algorithm MergeKLists(lists):
  minHeap = []
  for list in lists:
    if list.head not null:
      heappush(minHeap, (list.head.val, list.head))
  result = null
  current = null
```

```
  while minHeap not empty:
    val, node = heappop(minHeap)
    if result null:
      result = current = node
    else:
      current.next = node
      current = node
    if node.next:
      heappush(minHeap, (node.next.val, node.next))
  return result
```

Time: $O(nk \log k)$ (each element heap operation $\log k$). Space: $O(k)$ heap.

Problem: Longest Substring Without Repeating Characters

Find longest substring with no duplicate characters.

Example: ”abcabcbb”. LIS: ”abc” (length 3).

Approach: Sliding window. Maintain set of characters in current window. Expand window by adding characters, shrink if duplicate found.

```
Algorithm LongestSubstring(s):
  char_index = {}
  max_length = 0
  start = 0
  for end = 0 to len(s)-1:
    if s[end] in char_index:
      start = max(start, char_index[s[end]] + 1)
    char_index[s[end]] = end
    max_length = max(max_length, end - start + 1)
  return max_length
```

Time: $O(n)$ (single pass). Space: $O(\min(n, alphabet))$.

Example trace: ”abcabcbb”.

start=0, end=0: 'a' at 0, max$_length = 1$.

start=0, end=1: 'b' at 1, max$_length = 2$.

start=0, end=2: 'c' at 2, max$_length = 3$.

start=0, end=3: 'a' at 0, start=1 (move past previous 'a'), max$_length = 3$.

(Continue until no updates to max$_length$).

Final: max$_length = 3(”abc”)$.

Interview Preparation

Many companies assess algorithm knowledge via coding interviews.

Preparation:

1. Know classic algorithms (sorting, searching, graph traversal, dynamic programming).

2. Code frequently. Implement algorithms from scratch, handle edge cases.

3. Practice problems: LeetCode, HackerRank, Codeforces.

4. Communicate clearly. Explain approach before coding. Walk through examples.

5. Optimize iteratively. Start correct, then optimize if necessary.

6. Test thoroughly. Test normal, edge, and stress cases.

Common interview questions:

- Reverse linked list (test pointer manipulation) - Merge sorted arrays (test merging technique) - Binary search variants (test search understanding) - Permutations/combinations (test backtracking) - Tree traversals (test tree understanding) - Graph problems (connectivity, paths) - Dynamic programming (knapsack, sequence alignment)

Algorithm Optimization Techniques

Once algorithm is correct, optimize:

Reduce constant factors: Loop unrolling, instruction-level parallelism, avoid function calls in tight loops.

Improve cache locality: Sequential access, block algorithms for data reuse.

Parallelize: Multi-threading, GPU acceleration, distributed computing.

Bit manipulation: Use bitmasks for subset iteration, faster than array allocation.

Precomputation: Compute answers offline, store in table, answer queries in $O(1)$.

Example: Segment tree preprocessing for range queries.

Algorithm Specialization by Domain

Different domains emphasize different algorithms:

Machine Learning: Gradient descent (optimization), matrix operations (linear algebra), graph algorithms (neural network training).

Databases: B-trees (indexing), hash tables (quick lookup), sorting (efficient aggregation).

Graphics: Convex hull, point location, Voronoi diagrams.

Game AI: Minimax with alpha-beta pruning, A* pathfinding, Monte Carlo tree search.

Cryptography: Modular exponentiation (RSA), elliptic curve algorithms, hash functions.

Networking: Shortest path (routing), maximum flow (congestion control), load balancing.

Final Reflections

Algorithm design combines problem-solving, mathematical reasoning, and engineering. Master algorithms to:

- Understand fundamental problem-solving techniques. - Recognize problem structures and applicable algorithms. - Implement

efficient solutions. - Analyze trade-offs and optimize appropriately.

Your journey in algorithms continues beyond this book. Study more advanced topics: approximation algorithms, parameterized complexity, online algorithms, quantum computing. Tackle real-world problems. Contribute innovations.

The field evolves as computational challenges grow. Invest in algorithmic knowledge—it compounds throughout your career.

Problem-Solving Framework: A Systematic Approach

When approaching a new problem, use this framework:

Step 1: Understand - Read problem multiple times. Clarify ambiguities. - Identify inputs, outputs, constraints, examples. - Estimate input size and feasible complexity. - Draw examples on paper.

Step 2: Brainstorm - List approaches: brute force, greedy, DP, divide-and-conquer, graphs, geometry. - For each approach, rough out time complexity. - Choose most promising approach based on constraints.

Step 3: Design - Sketch high-level algorithm. - Identify subproblems and dependencies. - Choose data structures. - Write pseudocode.

Step 4: Verify - Trace through provided examples. - Test edge cases: empty input, single element, all same, boundary values. - Reason about correctness informally (loop invariants, induction).

Step 5: Implement - Code carefully. Use meaningful variable names. - Handle edge cases explicitly. - Avoid off-by-one errors.

Step 6: Test - Run on examples and edge cases. - Check for integer overflow, division by zero, null pointers. - Estimate time complexity; verify against limits.

Step 7: Optimize - Profile to identify bottlenecks (if needed). - Optimize data structures or algorithm (if needed). - Balance readability vs. optimization.

Step 8: Review - Are there simpler or more elegant solutions? - Can you simplify the code? - Document non-obvious parts.

Common Algorithmic Insights

After solving many problems, certain insights emerge:

Insight 1: Two pointers often beat nested loops.

For two-pointer patterns (sorted array, linked list), single pass with two pointers often beats $O(n^2)$ nested loop.

Example: Sum pairs equaling target. Nested loop: $O(n^2)$. Two pointers: $O(n)$ after sorting.

Insight 2: Hashing trades space for time.

Storing data in hash table enables $O(1)$ lookup vs. $O(\log n)$ binary search or $O(n)$ linear search.

Example: Valid anagram. Hash-based: $O(n)$. Sorting-based: $O(n \log n)$.

Insight 3: Sorting often simplifies problems.

Sorting reveals structure: duplicates become adjacent, order enables binary search, greedy becomes optimal.

Example: Merge intervals. Unsorted: quadratic. Sorted: linear.

Insight 4: Graphs model relationships.

Many problems reduce to graph problems. Treat as graph, apply BFS/DFS/shortest path.

Example: Word ladder. Model as graph, BFS finds shortest path.

Insight 5: DP handles overlapping subproblems.

If naive recursion recomputes subproblems, memoize or tabulate.

Example: Fibonacci. Naive: $O(2^n)$. DP: $O(n)$.

Insight 6: Greedy works with correct proof.

Greedy locally optimal choice leads to globally optimal only when greedy-choice and optimal-substructure properties hold.

Example: Activity selection. Greedy (choose earliest-ending) always optimal.

Insight 7: Binary search on answer.

If directly computing answer is hard but checking property $P(x)$ is easy, binary search for answer.

Example: Minimize maximum load. For each candidate maximum load, check if assignment possible (easy). Binary search.

Complexity Classes and Feasibility

Input size determines feasible complexity:

- $n \leq 10$: $O(n!)$ exhaustive search feasible. - $n \leq 20$: $O(2^n)$ DP or meet-in-middle feasible. - $n \leq 100$: $O(n^3)$ DP feasible. - $n \leq 10^3$: $O(n^2)$ nested loops feasible. - $n \leq 10^5$: $O(n \log n)$ sorting or advanced algorithms. - $n \leq 10^6$: $O(n)$ linear time required. - $n > 10^6$: Only $O(1)$ per element, strict $O(n)$ barely feasible.

Also consider time limits: 1 second $\approx 10^8$ operations on modern CPU. Adjust for constant factors.

Learning Progression

Advancing in algorithms:

Phase 1: Master fundamentals (sorting, searching, basic DP, graph algorithms).

Phase 2: Recognize problem patterns. Know when to apply which technique.

Phase 3: Optimize algorithms. Reduce constants, improve locality, parallelize.

Phase 4: Combine techniques. Use multiple algorithms in single solution.

Phase 5: Invent. Face novel problems, design new algorithms, prove correctness.

Most practitioners operate in Phases 1-3. Phase 4-5 requires deep expertise and experience.

Real-World Algorithmic Challenges

Theory provides tools; practice reveals challenges:

Approximate data: Real data has noise, outliers, missing values. Algorithms assume clean data.

Scale mismatches: Dataset larger than memory, too slow in practice despite theoretical efficiency.

Unknown properties: Data may not match assumptions (unsorted, non-uniform distribution).

Changing requirements: Requirements evolve; algorithm must adapt.

Maintenance: Code evolves. Optimized but unreadable code becomes liability.

Solutions:

- Test on realistic data (not just theory). - Profile to identify bottlenecks. - Communicate assumptions clearly. - Balance optimization and readability. - Build incrementally, refactor as needed.

This extended appendix provides deeper insights, systematic frameworks, and practical guidance for mastering algorithm design. Combined with the four modules on problem-solving paradigms, you have comprehensive foundation for algorithm study and practice.

Future Trends in Algorithms

Algorithm research continues evolving. Emerging areas:

Machine Learning Optimization: Stochastic gradient descent, Adam, distributed training for billion-parameter models. New theory understanding convergence, generalization.

Quantum Algorithms: Shor's factoring, Grover's search, potential speedups for specific problems. Currently limited by hardware.

Approximation and FPT: For hard problems, approximation guarantees and fixed-parameter tractability provide practical solutions.

Parallel and Distributed: Algorithms for multi-core, GPU, and distributed systems. New challenges: load balancing, communication minimization, fault tolerance.

Sublinear Algorithms: Process input in fewer than $O(n)$ time via sampling or streaming. Enable large-scale data analysis.

Data-Driven Algorithms: Incorporate machine learning into traditional algorithms. Examples: learned hashing, learned indexing, learned branching.

Competitive Online Algorithms: For problems where input arrives online, design algorithms competing with offline optimal.

Complexity Lower Bounds: Prove hardness results. Understanding barriers to improvement guides future research.

These areas offer exciting opportunities for research and innovation. The field is far from mature.

Conclusion

Algorithm design is a fundamental skill combining mathematics, computer science, and engineering. Mastering algorithms enables you to:

- Solve complex computational problems efficiently. - Understand trade-offs and optimize appropriately. - Recognize when problems are related (reduce one to another). - Design novel algorithms for new problems. - Communicate solutions clearly.

Throughout this book, we've explored problem-solving paradigms, analyzed algorithm complexity, examined specific algorithms, and developed frameworks for approaching novel problems.

Your journey in algorithms is just beginning. Continue learning: study new techniques, solve challenging problems, implement algorithms, contribute to open-source, and perhaps invent novel algorithms.

Algorithm expertise compounds. Early investment in understanding provides lifetime benefits. Every problem-solving skill strengthens your algorithmic thinking.

Go forth, design elegant algorithms, and solve important problems.

Glossary

Amortized: Average cost accounting for expensive operations amortized over many operations.

Backtracking: Explore solution space, undo choices leading to dead ends.

Memoization: Cache subproblem solutions to avoid recomputation.

Greedy: Make locally optimal choices hoping for global optimality.

Heuristic: Problem-solving technique without guarantee of optimality.

In-place: Algorithm modifying input without extra proportional storage.

Invariant: Property maintained throughout algorithm execution.

Polynomial: Algorithm with time $O(n^k)$ for fixed k.

Pseudo-random: Deterministic algorithm producing random-looking values.

Reduction: Transform instance of problem to instance of another problem.

Robustness: Algorithm correctly handles edge cases and invalid inputs.

Stability: Sorting preserving relative order of equal elements.

Streaming: Process data as stream without storing all.

Subproblem: Smaller instance of original problem.

Trade-off: Optimize one objective at expense of another.

Worst-case: Maximum time over all inputs of given size.

Zero-indexed: Array indices start at 0 (common in programming).

ZString: Precomputed string matching structure (Z-algorithm).

Hash collision: Multiple keys mapping to same hash value.

Amortized Analysis: Average-case analysis of sequence of operations.

Bibliography

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

2. Kleinberg, J., & Tardos, É. (2005). *Algorithm Design*. Pearson.

3. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

4. Skiena, S. S. (2008). *The Algorithm Design Manual* (2nd ed.). Springer.

5. Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983). *Data Structures and Algorithms*. Addison-Wesley.

6. Horowitz, E., Sahni, S., & Anderson, G. (2008). *Fundamentals of Computer Algorithms*. Silicon Press.

7. Mehlhorn, K., & Sanders, P. (2008). *Algorithms and Data Structures*. Springer.

8. Roughgarden, T. (2017). *Algorithms Illuminated*. Soundlikeyourself Productions.